ASSIGNMENT 2

Design of a Vector Embedding for Capability Composition

Formal representations, compatibility, and compositional reasoning

Technical Report

Problem-specific vector representation for formally specified application states, goals, and executable capabilities.

Implementation status: Working experimental system with five required experiments and generated result figures.

Abstract

This report presents a problem-specific vector representation for formally specified application states, goals, and executable capabilities. The design combines categorical execution type, structural attributes, hashed semantic features, and operational attributes in a fixed-dimensional capability vector. Compatibility is evaluated symbolically through precondition–effect and output–input relationships, while cosine-based similarity is used to compare encoded capabilities. A composition operator constructs a composite capability from a valid sequence and aggregates its effects, resources, constraints, costs, risk, reliability, and availability. Five experiments evaluate compatibility, composition, alternative implementations, goal relevance, and operational attributes. The observed results show that the representation distinguishes compatible from incompatible capability pairs, produces non-identical relationships between a composite and its components, captures functional relationships among different implementation mechanisms, separates goal-relevant from irrelevant capabilities, and reflects operational differences between alternative payment capabilities.

Keywords: capability embedding, vector representation, capability composition, compatibility, preconditions, effects, input-output matching, goal relevance, operational attributes

Contents

1. Problem Definition
2. Design Requirements
3. Related Embedding Approaches
4. Proposed Representation
5. Mathematical Formulation
6. Capability Composition Model
7. Implementation
8. Experimental Methodology
9. Results
10. Analysis
11. Limitations
12. Conclusion

Repository artifacts produced by the implementation

- Source modules for models, embedding, compatibility, and composition.
- A JSON experimental dataset containing the initial state, goal, capabilities, constraints, resources, and operational attributes.
- A runnable experiment script producing a CSV result file and five figures.
- This technical report, which documents the formal design, implementation, methodology, results, analysis, and limitations.

1. Problem Definition

The assignment considers a formally specified application rather than an application described only in natural language. The application is modeled as A = (S, C, SI, G, R, K), where S is the state space, C is the set of available capabilities, SI is the initial state, G is the goal specification, R is the resource set, and K contains global constraints and policies.

The central problem is to design a vector space in which the structure needed for capability composition is preserved. A capability is not merely a label. It has a type, inputs, outputs, preconditions, effects, constraints, resource requirements, operational attributes, and an execution mechanism. Two capabilities may therefore be semantically related while differing in implementation or operational behavior.

The representative application scenario is an order-processing workflow. The initial state contains an authenticated customer, an existing cart with three items, no existing order, a payment status of NOT_STARTED, available inventory, and an unsent notification. The target goal requires an existing order, successful payment, and a sent notification.

The required research question is whether a suitably designed vector representation can preserve the relationships required for compatibility, composition, goal relevance, and operational reasoning.

Scope boundary

The assignment explicitly focuses on representation rather than redesigning the planning or replanning algorithm. Therefore, the implementation does not attempt to implement BFS, DFS, A*, D* Lite, LPA*, or a new planner. The planner is treated as background context; the work here represents the operations that a planner could compose.

2. Design Requirements

The design was developed to satisfy the following requirements.

- **Capability identity:** distinct capabilities should remain distinguishable in the vector space.
- **State awareness:** states and goals must be encodable so that capability effects and desired conditions can be represented.
- **Precondition–effect compatibility:** the system must identify whether effects produced by one capability satisfy preconditions of another.
- **Input–output compatibility:** outputs of one capability should be matchable to required inputs of another.
- **Similarity versus composability:** high vector similarity must not be treated as a substitute for symbolic executability.
- **Composition:** valid atomic capabilities must be combinable into a composite representation.
- **Goal relevance:** capabilities contributing to the specified goal should be distinguishable from irrelevant capabilities.
- **Operational properties:** at least two operational attributes must influence the representation or its evaluation. This implementation includes time, resource cost, monetary cost, risk, reliability, and availability.
- **Implementation:** the system must support encode(state), encode(goal), encode(capability), compose(capabilities), and similarity(x,y).
- **Efficiency:** the representation should be fixed-dimensional and inexpensive to compute for the small structured dataset used in the experiment.

A key design decision is to combine symbolic reasoning with vector similarity. Symbolic checks are used where exact logical relationships matter, especially compatibility and goal relevance. Vectors are used where graded similarity is informative, such as comparing alternative implementations or relating a composite capability to its components.

3. Related Embedding Approaches

Several representation families are relevant to this problem. A one-hot representation can encode categorical capability types clearly, but it does not by itself capture structured relationships among inputs, outputs, preconditions, effects, resources, and operational values. Traditional feature engineering can explicitly represent these attributes, but the representation must be designed for the application and can become sparse as the vocabulary grows.

Distributional and neural embeddings, such as word or sentence embeddings, provide dense vector representations and useful notions of semantic similarity. However, a generic language embedding does not inherently guarantee exact preservation of symbolic precondition–effect or input–output constraints. For this reason, directly applying a pretrained text embedding would not be sufficient for the present assignment.

The proposed approach therefore adopts a hybrid feature-based design. Categorical information is represented explicitly, structured fields are converted into deterministic feature keys, semantic tokens are mapped into a fixed hashed feature space, and operational attributes are represented numerically. Compatibility remains a symbolic operation rather than being inferred only from cosine similarity.

Approach	Strength	Limitation for this problem
One-hot / categorical	Clear identity and implementation type	Weak relational semantics
Hand-crafted structured features	Directly represents domain constraints	Requires problem-specific design
Generic text embeddings	Dense semantic similarity	No guarantee of exact symbolic compatibility
Hybrid structured vector (proposed)	Combines identity, structure, semantics, and operations	Requires explicit feature design and weighting

4. Proposed Representation

4.1 State representation

A state is represented as a mapping from state-variable keys to values. The encoder converts deterministic key–value tokens into a fixed 24-dimensional vector using a stable hashing scheme. This representation supports Boolean, numeric, and categorical state values without requiring a learned model.

4.2 Goal representation

A goal is encoded using the same key–value mechanism as a state, also producing a 24-dimensional vector. The goal encoder therefore provides a compact representation of desired conditions while preserving the variable names and values through deterministic feature keys.

4.3 Capability representation

The capability model follows the assignment's formal structure: type, inputs, outputs, preconditions, effects, constraints, resources, cost/quality attributes, reliability, availability, and execution mechanism. The implemented vector has 37 dimensions.

Block	Dimensions	Purpose
Capability type	9	One-hot representation of API, DATABASE, GUI, EVENT, FUNCTION, FILE, COMPUTATION, MESSAGE, SERVICE
Structural features	6	Compact counts/structural properties of inputs, outputs, preconditions, effects, constraints, and resources
Semantic hashed features	16	Fixed-size representation of field/value keys and structured content
Operational features	6	Time, resource cost, monetary cost, risk, reliability, availability
Total capability vector	37	Concatenation of all blocks

The design intentionally keeps implementation type visible. Consequently, two capabilities can have similar effects while remaining distinguishable as API, DATABASE, or GUI implementations.

5. Mathematical Formulation

5.1 Formal capability

For each capability Ci, the structured representation is:

$$Ci = (Ti, Ii, Oi, Pi, Ei, Ki, Ri, Qi, Reli, Ai, Mi)$$

where Ti is the capability type; Ii and Oi are inputs and outputs; Pi and Ei are preconditions and effects; Ki is the constraint set; Ri is the resource requirement; Qi contains cost/quality attributes; Reli is reliability; Ai is availability; and Mi is the execution mechanism.

5.2 Capability embedding

The 37-dimensional capability embedding is written as:

$$E(Ci) = [\ T(Ci) \ ||\ Fstruct(Ci) \ ||\ Fsem(Ci) \ ||\ Fop(Ci)\ ] \in R^{37}$$

T is the 9-dimensional type vector, Fstruct is the 6-dimensional structural block, Fsem is the 16-dimensional hashed semantic block, and Fop is the 6-dimensional operational block.

5.3 Compatibility

For a pair (Ci, Cj), the implementation computes two symbolic scores. The first measures how many preconditions of Cj are satisfied by effects of Ci. The second measures whether outputs of Ci satisfy required inputs of Cj.

$$Compat(Ci,Cj) = 0.60 \cdot PScore(Ci,Cj) + 0.40 \cdot IOScore(Ci,Cj)$$

A pair is considered compatible when the resulting score is at least 0.5. This threshold is used in the composition operation. The distinction is important: similarity is graded, while compatibility is a logical/operational decision.

5.4 Similarity

Vector similarity is based on cosine similarity. The final capability similarity combines functional vector similarity with a separately computed operational similarity:

$$Sim(Ci,Cj) = 0.70 \cdot Cos(E(Ci),E(Cj)) + 0.30 \cdot OpSim(Ci,Cj)$$

Operational similarity is one minus the mean normalized absolute difference across the six operational features. Normalization prevents time, resource, monetary cost, risk, reliability, and availability from being compared directly on incompatible scales.

6. Capability Composition Model

The assignment defines a valid two-step composition as C1:S0→S1 followed by C2:S1→S2, provided that the outputs and effects of C1 satisfy the inputs and preconditions of C2. The implementation extends this rule recursively to longer sequences.

For a valid sequence C1, C2, ..., Cn:

$$C1 \rightarrow C2 \rightarrow \ldots \rightarrow Cn \Rightarrow Ccomp$$

The composite capability inherits the input requirements and preconditions of the first capability and the outputs and effects of the last capability. Constraints are concatenated, resources are combined as a union, and operational attributes are aggregated.

Composite attribute	Aggregation rule
Inputs	Inputs of the first capability
Preconditions	Preconditions of the first capability
Outputs	Outputs of the last capability
Effects	Effects of the last capability
Constraints	Concatenation of component constraints
Resources	Union of component resources
Time/resource/money cost	Sum across components
Risk	Sum across components
Reliability	Product across components
Availability	Product across components
Execution	Sequential execution of components

For the main composition CreateOrder → MakePayment → SendNotification, the resulting composite capability has the final effect Notification.sent = true. Its reliability is the product 0.99 × 0.99 × 0.98 = 0.960498, which is the value reported by the implementation.

7. Implementation

The implementation is written in Python and is intentionally small. It is organized into separate modules so that the formal model, embedding, compatibility, and composition logic remain independently testable.

File	Responsibility
src/models.py	Dataclasses for State, Goal, and Capability
src/embedding.py	State/goal encoding, capability encoding, cosine similarity, operational similarity
src/compatibility.py	Precondition–effect and output–input compatibility scoring
src/composition.py	Validation and construction of composite capabilities
data/dataset.json	Initial state, goal, atomic capabilities, constraints, resources, operational values
experiments/run_all.py	Runs all five experiments and generates CSV and figures
results/results.csv	Machine-readable experimental results
results/figures/	Five experiment plots

Required interface

The implementation supports the assignment's requested operations through the embedding and composition classes/functions: state encoding, goal encoding, capability encoding, capability composition, and similarity comparison.

The experiment runner also records the exact numerical results to CSV, making the plots reproducible from the same execution. The successful run produced a 37-dimensional capability representation and 24-dimensional state and goal representations.

8. Experimental Methodology

Five experiments were constructed directly from the formal capability model and the required patterns in the assignment. The dataset contains the order-processing state and goal, the three capabilities needed for the main composition, incompatible and irrelevant capabilities, and alternative payment implementations with different operational properties.

Experiment	Question	Expected observation
1. Compatibility	Can compatible and incompatible pairs be distinguished?	CreateOrder→MakePayment compatible; CreateOrder→CancelCart incompatible
2. Composition	How does a composite relate to its atomic components?	Composite should have meaningful, non-identical similarities to C1, C2, C3
3. Alternative implementations	Can similar functions remain distinguishable by implementation?	API, DATABASE, and GUI should be related but not identical
4. Irrelevant capabilities	Can goal-relevant and irrelevant capabilities be separated?	Order workflow capabilities relevant; PlayMusic irrelevant
5. Operational attributes	Do operational differences affect representation?	FastPayment and SlowPayment remain functionally related but differ operationally

Evaluation measures

- Compatibility score and compatibility decision.
- Cosine-based similarity between a composite and each atomic component.
- Pairwise similarity among API, DATABASE, and GUI implementations.
- Binary goal-relevance score derived from goal-effect contribution.
- Combined and operational-only similarity for alternative payment capabilities.

9. Results

9.1 Experiment 1 — Capability compatibility

The compatible pair obtained 1.000000, while the incompatible pair obtained 0.400000. Since the composition threshold is 0.5, the first pair is accepted and the second is rejected.

Pair	Compatibility score	Decision
CreateOrder → MakePayment	1.000000	Compatible
CreateOrder → CancelCart	0.400000	Incompatible

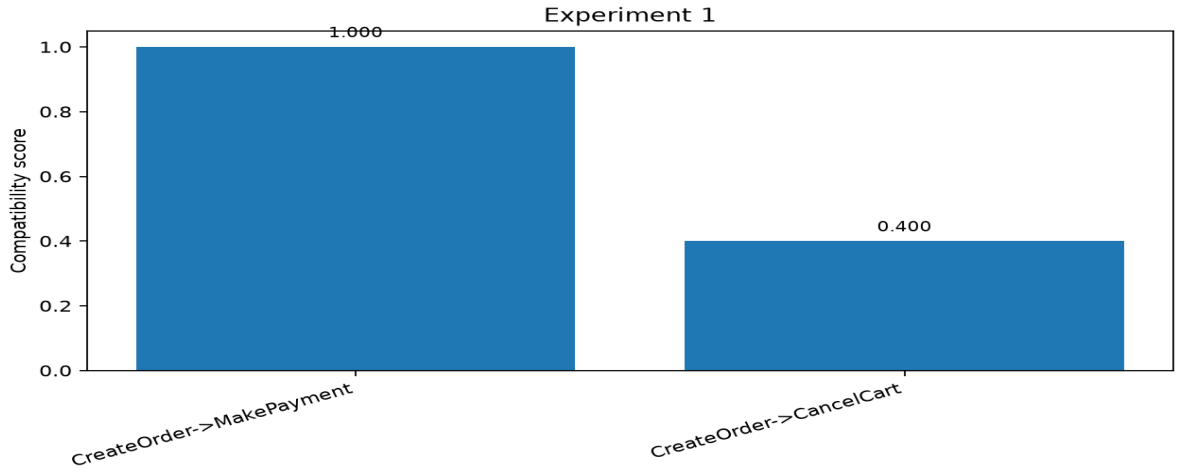


Figure 1. Experiment 1 compatibility scores.

9.2 Experiment 2 — Capability composition

The composite CreateOrder → MakePayment → SendNotification was successfully constructed. The resulting similarities were 0.742610, 0.830463, and 0.703753 for CreateOrder, MakePayment, and SendNotification respectively.

Comparison	Similarity
Composite ↔ CreateOrder	0.742610
Composite ↔ MakePayment	0.830463
Composite ↔ SendNotification	0.703753

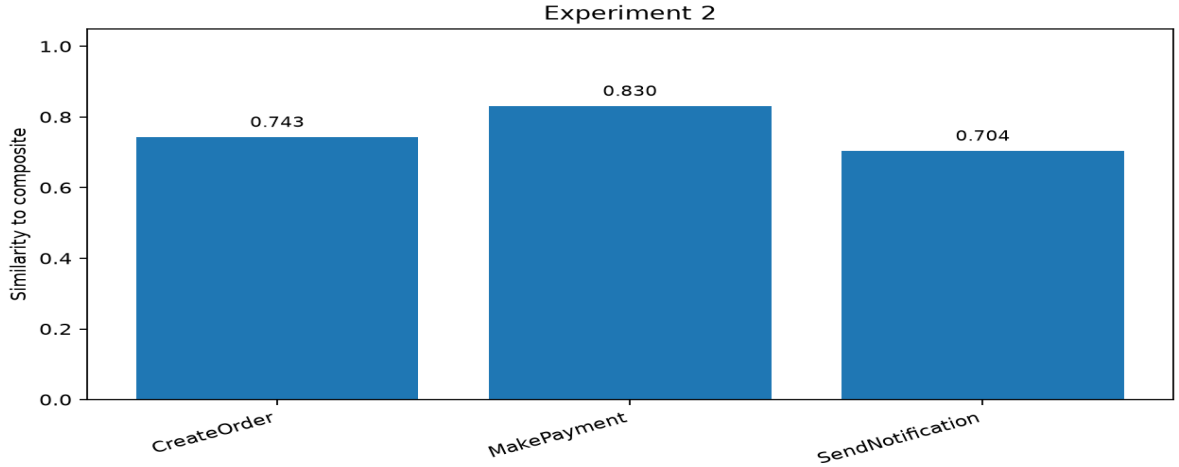


Figure 2. Experiment 2 similarity of the composite to its components.

9.3 Experiment 3 — Alternative implementations

The three implementation mechanisms produced different pairwise similarities: API–DATABASE = 0.741516, API–GUI = 0.864717, and DATABASE–GUI = 0.700885. This shows that the representations are related but not numerically identical.

Pair	Similarity
API ↔ DATABASE	0.741516
API ↔ GUI	0.864717
DATABASE ↔ GUI	0.700885

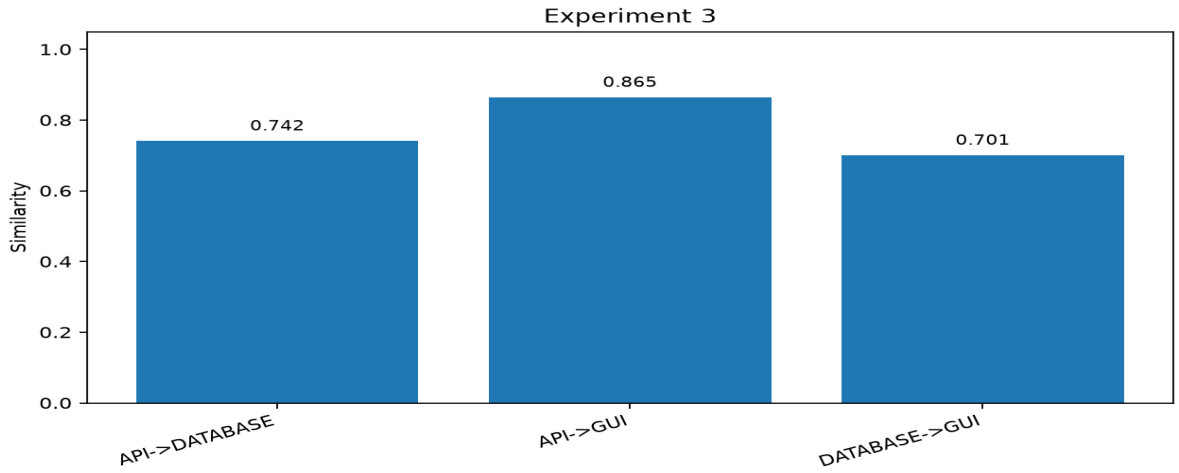


Figure 3. Experiment 3 alternative implementation similarities.

9.4 Experiment 4 — Irrelevant capabilities

CreateOrder, MakePayment, and SendNotification each obtained a goal-relevance score of 1.0, while PlayMusic obtained 0.0. The representation therefore separates capabilities contributing to the order goal from a capability unrelated to that goal.

Capability	Goal relevance
CreateOrder	1.000
MakePayment	1.000
SendNotification	1.000
PlayMusic	0.000

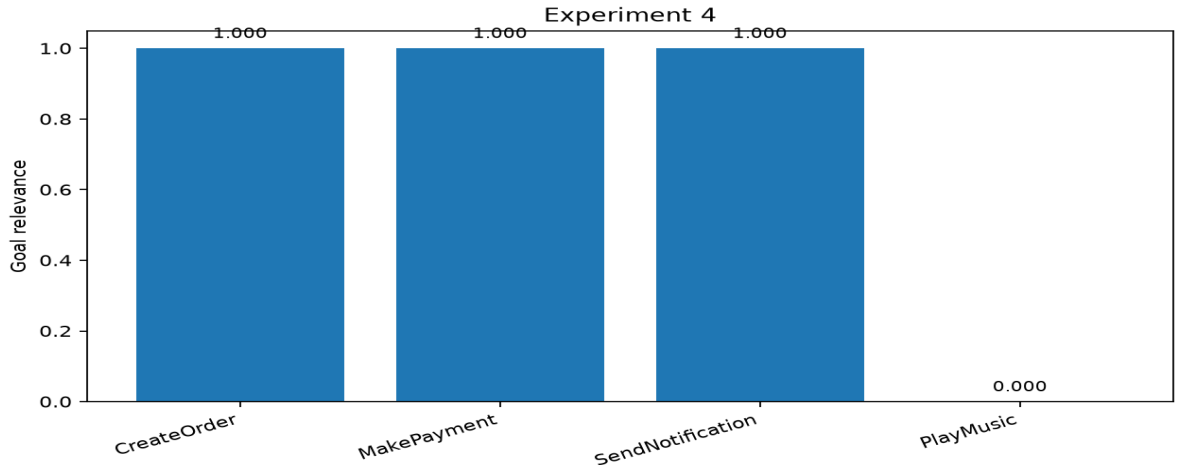


Figure 4. Experiment 4 goal relevance.

9.5 Experiment 5 — Operational attributes

FastPayment and SlowPayment produced a combined similarity of 0.789137 and an operational-only similarity of 0.578333. The lower operational similarity indicates that their operational characteristics differ despite their common payment function.

Measure	Value
Combined similarity	0.789137
Operational similarity	0.578333

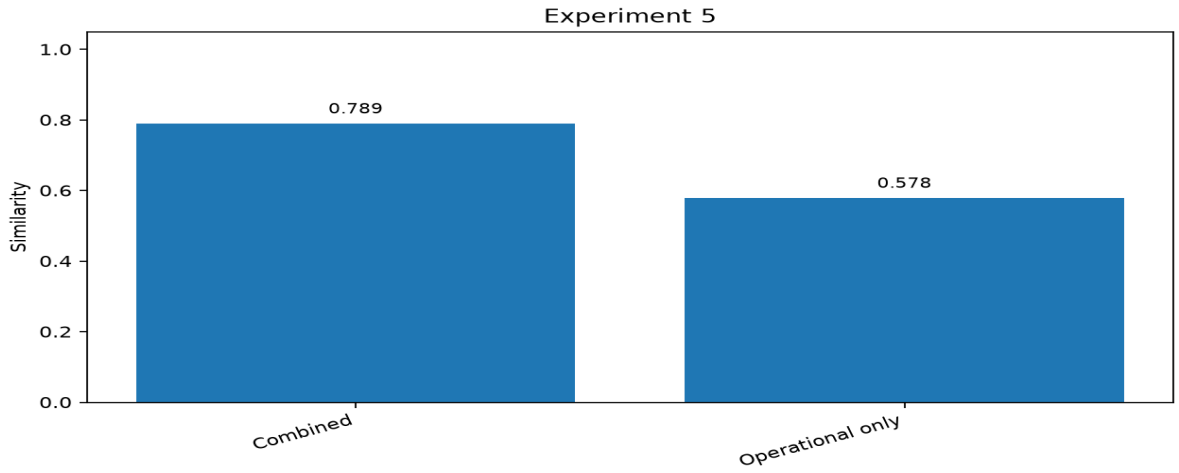


Figure 5. Experiment 5 combined and operational similarity.

10. Analysis

10.1 Capability compatibility

Experiment 1 provides direct evidence that symbolic compatibility can distinguish the required cases. The score of 1.0 for CreateOrder → MakePayment follows from matching Order.exists = true and the order identifier dependency. The 0.4 score for CreateOrder → CancelCart is below the 0.5 threshold because CancelCart requires Order.exists = false while CreateOrder produces Order.exists = true. Thus, the system does not incorrectly compose the two capabilities.

10.2 Composition behavior

Experiment 2 demonstrates that the composite is not simply assigned an identical vector to every component. The similarity values differ across the three components. MakePayment has the highest similarity (0.830463), while SendNotification has the lowest (0.703753). This variation provides a more informative representation than forcing every constituent to have the same similarity.

10.3 Alternative implementations

Experiment 3 shows that implementation type remains visible in the representation. API, DATABASE, and GUI capabilities can therefore be functionally related without collapsing into a single identical vector. The pairwise values range from 0.700885 to 0.864717, which indicates relatedness with implementation-sensitive distinctions.

10.4 Goal relevance

Experiment 4 gives a clear separation between capabilities that contribute to the stated order goal and the unrelated PlayMusic capability. This result is based on matching capability effects to goal conditions rather than assuming that vector similarity alone can establish goal achievement.

10.5 Operational properties

Experiment 5 demonstrates why operational information should not be discarded. FastPayment and SlowPayment have the same broad functional role but differ in time, resource requirement, monetary cost, risk, reliability, and availability. The operational-only similarity of 0.578333 is lower than the combined similarity of 0.789137, showing that operational differences contribute additional information beyond functional identity.

10.6 Overall evaluation

Evaluation criterion	Assessment from experiments
Capability representation	Different capabilities receive structured, fixed-dimensional vectors.
State relationship	States and goals have explicit encoders and capability effects are represented.
Precondition–effect compatibility	Distinguished successfully in Experiment 1.
Input–output compatibility	Included explicitly in the compatibility score and composition validation.
Composition	Three compatible capabilities were composed successfully.
Goal relevance	Relevant and irrelevant capabilities were separated in Experiment 4.
Operational properties	Six operational attributes are encoded and evaluated.
Consistency	All five experiments run successfully from the same dataset and encoder.
Efficiency	Fixed-dimensional vectors and direct feature operations avoid expensive model inference.

11. Limitations

The proposed system is intentionally a problem-specific experimental representation rather than a learned universal embedding model. Several limitations should therefore be recognized.

- **Hash-based semantic representation:** deterministic hashing provides a compact fixed dimension, but unrelated feature keys can collide.
- **Hand-selected weights:** the 0.70/0.30 similarity weighting and the 0.60/0.40 compatibility weighting are design choices rather than learned from a benchmark.
- **Symbolic goal relevance:** the current relevance experiment uses explicit effect/goal matching and does not learn graded goal usefulness from data.
- **Small dataset:** the experiments use a compact order-processing scenario and a small set of capabilities. Broader datasets are needed to establish generalization.
- **Operational assumptions:** operational attributes are normalized using fixed ranges. Different application domains may require different scales.
- **No learned composition operator:** composition is explicitly constructed from structured capability fields rather than learned from large numbers of examples.
- **Availability simplification:** availability is represented numerically in the implemented vector rather than as a time-dependent availability function.

These limitations do not invalidate the required experiments. Instead, they identify the main areas for future work if the representation is extended to larger, heterogeneous application domains.

12. Conclusion

This work designed and implemented a problem-specific vector representation for formally specified application states, goals, and executable capabilities. The representation combines capability type, structural information, deterministic semantic features, and operational attributes in a fixed 37-dimensional capability vector, with 24-dimensional state and goal encodings.

The five required experiments provide evidence that the design captures the relationships needed for capability composition. Experiment 1 distinguishes a valid CreateOrder → MakePayment transition from the incompatible CreateOrder → CancelCart transition. Experiment 2 constructs a three-capability composite and obtains meaningful, non-identical similarities to its components. Experiment 3 preserves relationships among API, DATABASE, and GUI implementations without representing them identically. Experiment 4 distinguishes goal-relevant capabilities from an irrelevant capability. Experiment 5 shows that operational differences can be represented separately from functional similarity.

The main design principle is that vector similarity and symbolic compatibility serve different purposes. Vectors provide graded representations and similarity, while explicit precondition/effect and input/output checks provide the logical guarantees required for safe composition. This hybrid approach is therefore more appropriate for the assignment than treating a generic semantic embedding as the complete solution.

Overall, the implementation satisfies the assignment's required formal representation, implementation, experimental dataset, and experimental evaluation objectives. The resulting system provides a concrete foundation for future work on learned capability embeddings, larger application datasets, and automated planning over composite capability representations.